

Python for AI & Machine Learning

Class 18 – File Handling in Python

Course: Python for AI & Machine Learning

Module: Advanced Python Programming

Duration: 2 Hours (Theory + Practical)

Learning Objectives

By the end of this session, students will be able to:

- Understand the importance of File Handling.
 - Read data from text files.
 - Write data to files.
 - Append data to existing files.
 - Understand different file modes.
 - Work with CSV and JSON files.
 - Handle exceptions while working with files.
 - Apply File Handling in AI & Machine Learning projects.
-

Revision

In the previous class, we learned:

- Introduction to Exceptions
- Error vs Exception
- Built-in Exceptions
- `try`, `except`, `else`, and `finally`
- Raising Exceptions
- Custom Exceptions

Today, we will learn how Python stores and retrieves data using **File Handling**.

Introduction to File Handling

A **File** is a collection of data stored permanently on a storage device such as a hard disk or SSD.

Python allows us to:

- Create files
- Read files

- Write files
- Update files
- Delete files

Without File Handling, data is stored only in memory and is lost when the program ends.

Why Do We Need File Handling?

Suppose you build:

- Student Management System
- Employee Management System
- AI Chatbot
- Banking Application
- Hospital Management System

All these applications need to store information permanently.

File Handling makes this possible.

Types of Files

Python mainly works with two types of files.

1. Text Files

Examples:

- .txt
- .csv
- .json
- .xml

These files store readable text.

2. Binary Files

Examples:

- Images
- Videos
- Audio
- Machine Learning Models (.pkl)
- PDF Files

Binary files store data in binary format.

Opening a File

Python uses the `open()` function.

Syntax

```
file = open("filename", "mode")
```

Example:

```
file = open("students.txt", "r")
```

File Modes

Mode	Description
<code>r</code>	Read existing file
<code>w</code>	Write (creates a new file or overwrites an existing file)
<code>a</code>	Append data to an existing file
<code>x</code>	Create a new file (fails if file already exists)
<code>rb</code>	Read binary file
<code>wb</code>	Write binary file
<code>r+</code>	Read and write
<code>w+</code>	Write and read
<code>a+</code>	Append and read

Reading a File

Method 1 – `read()`

Reads the complete file.

```
file = open("students.txt", "r")  
  
content = file.read()
```

```
print(content)

file.close()
```

Method 2 – `readline()`

Reads one line at a time.

```
file = open("students.txt", "r")

print(file.readline())

file.close()
```

Method 3 – `readlines()`

Reads all lines and returns a list.

```
file = open("students.txt", "r")

lines = file.readlines()

print(lines)

file.close()
```

Writing to a File

The `w` mode creates a new file if it does not exist.

If the file already exists, its contents are overwritten.

```
file = open("students.txt", "w")

file.write("Rahul\n")
file.write("Priya\n")
file.write("Aman\n")

file.close()
```

Appending Data

The `a` mode adds new data without removing existing data.

```
file = open("students.txt", "a")  
  
file.write("Sneha\n")  
  
file.close()
```

Closing a File

Always close a file after use.

```
file.close()
```

Closing a file:

- Saves changes.
- Releases system resources.
- Prevents data corruption.

Using the `with` Statement

Instead of manually closing a file, Python provides the `with` statement.

```
with open("students.txt", "r") as file:  
    print(file.read())
```

Advantages:

- Automatically closes the file.
- Cleaner code.
- Safer programming practice.

Working with CSV Files

CSV stands for **Comma-Separated Values**.

Example:

```
Name, Age, Course
Rahul, 22, Python
Sneha, 23, AI
```

Reading a CSV file:

```
import csv

with open("students.csv", "r") as file:
    reader = csv.reader(file)

    for row in reader:
        print(row)
```

Writing a CSV File

```
import csv

with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)

    writer.writerow(["Name", "Age", "Course"])
    writer.writerow(["Rahul", 22, "Python"])
    writer.writerow(["Sneha", 23, "AI"])
```

Working with JSON Files

JSON stands for **JavaScript Object Notation**.

Example:

```
{
    "name": "Rahul",
    "course": "Python",
    "age": 22
}
```

Writing JSON:

```
import json

student = {
    "name": "Rahul",
    "course": "Python",
    "age": 22
}

with open("student.json", "w") as file:
    json.dump(student, file)
```

Reading JSON

```
import json

with open("student.json", "r") as file:
    student = json.load(file)

print(student)
```

Exception Handling with Files

```
try:

    with open("students.txt", "r") as file:
        print(file.read())

except FileNotFoundError:
    print("File does not exist.")
```

AI Example – Reading Dataset

```
import csv

with open("dataset.csv", "r") as file:

    reader = csv.reader(file)
```

```
for row in reader:
    print(row)
```

Machine Learning projects commonly read datasets from CSV files.

AI Example – Saving Model Predictions

```
with open("predictions.txt", "a") as file:

    file.write("Prediction: Spam\n")
```

Predictions generated by AI models can be stored in files for later analysis.

AI Example – Saving Configuration

```
import json

config = {
    "model": "Linear Regression",
    "learning_rate": 0.01
}

with open("config.json", "w") as file:
    json.dump(config, file)
```

JSON files are often used to store model configurations.

Best Practices

- Always use the `with` statement.
 - Close files properly if not using `with`.
 - Handle `FileNotFoundError`.
 - Use appropriate file modes.
 - Validate file paths before opening files.
 - Store structured data using CSV or JSON.
-

Practical Exercise 1

Create a text file named `students.txt`.

Write the names of five students.

Read and display the contents.

Practical Exercise 2

Append two new student names to the existing file.

Display the updated contents.

Practical Exercise 3

Create a CSV file containing:

- Name
- Age
- Course

Read the file and display all records.

Practical Exercise 4

Create a JSON file storing:

- Name
- Age
- Skills
- Course

Read the JSON file and display the data.

Practical Exercise 5

Read a file using the `with` statement.

Handle `FileNotFoundError`.

Assignment

Student Record Management System

Develop a **Student Record Management System** using File Handling.

Requirements

The program should:

- Add student records.
 - View student records.
 - Append new records.
 - Store records in a CSV file.
 - Store application settings in a JSON file.
 - Handle file-related exceptions.
 - Display appropriate messages when files are missing.
-

Interview Questions

1. What is File Handling?
 2. Why is File Handling important?
 3. What are the different file modes in Python?
 4. What is the difference between `read()`, `readline()`, and `readlines()`?
 5. What is the purpose of the `with` statement?
 6. What is the difference between text files and binary files?
 7. What is a CSV file?
 8. What is JSON?
 9. How do you handle `FileNotFoundError`?
 10. How is File Handling used in AI and Machine Learning projects?
-

Summary

In today's session, we learned:

- Introduction to File Handling
- Types of Files
- Opening and Closing Files
- File Modes
- Reading Files
- Writing Files
- Appending Data
- Using the `with` Statement
- Working with CSV Files
- Working with JSON Files

- Exception Handling with Files
- AI & Machine Learning examples
- Best Practices
- Practical Exercises
- Assignment

File Handling is a fundamental skill for Python developers. It enables applications to store, retrieve, and process data efficiently, making it essential for real-world software development and AI & Machine Learning projects.

Next Class Preview

Class 19 – Modules and Packages in Python

Topics to be covered:

- Introduction to Modules
- Creating Your Own Module
- Importing Modules
- The `math` Module
- The `random` Module
- The `datetime` Module
- Creating Packages
- Installing Third-Party Libraries using `pip`
- Introduction to NumPy and Pandas
- Real-world Applications in AI & Machine Learning